

CSCE 676 DATA MINING · MODULE 03 · LECTURE NOTES

Frequent Patterns

Co-occurrence, and the rules that do not mean what they look like.

MODULE OUTCOMES

- Explain support, confidence, and lift.
- Explain the monotonicity property and the algorithms using it.
- Explain why confidence is misleading.
- Explain the multiplicity problem in this setting.
- Mine and assess a rule set.

1 The measures

$$\text{support}(X) = P(X) \quad \text{conf}(X \rightarrow Y) = P(Y|X) \quad \text{lift}(X \rightarrow Y) = P(Y|X) / P(Y)$$

- **Support is the fraction of transactions containing X** — a frequency, and **the quantity the algorithms prune on** (§2), which makes it structurally central as well as descriptively useful.
- **Confidence is how often Y appears given X** — intuitive, and misleading on its own, which is §1's callout and §3's warning.
- **Lift is confidence relative to Y 's base rate. Lift = 1 means no association at all**; above 1 means Y is more likely given X ; below 1 means less likely.
- **Lift is the one that accounts for the base rate** — and **confidence without it is the standard error in this material**, which is why this module leads with lift rather than introducing it as a refinement.

HIGH CONFIDENCE CAN MEAN NOTHING AT ALL, BECAUSE Y MAY SIMPLY BE COMMON

If 80% of all transactions contain bread, then *any* rule of the form $X \rightarrow \text{bread}$ has confidence near 0.8 — including rules where X is entirely unrelated to bread, because confidence is just the conditional frequency.

So the rule 'customers who buy caviar also buy bread, with 80% confidence' is simultaneously true and completely uninformative — its lift is 1.0, which means knowing about the caviar tells you nothing whatsoever about the bread.

Which is why lift, or a comparable base-rate-adjusted measure, is necessary rather than optional — and why a rule list sorted by confidence is dominated by whatever items are most popular, which is both useless and reliably convincing-looking.

And lift has its own problem worth stating: it is unstable for rare items, where a handful of co-occurrences produces an enormous ratio with no real support behind it — so report support alongside it, always, and treat a high-lift low-support rule as noise until shown otherwise.

2 The algorithms

THE DOWNWARD CLOSURE PROPERTY

if an itemset is frequent, every subset of it is frequent. Equivalently: if a set is infrequent, then every superset of it is infrequent too.

WHICH PRUNES ENORMOUSLY

a candidate set with any infrequent subset can be discarded without ever being counted.

APRIORI

level by level: generate candidates of size $k+1$ only from the frequent sets of size k , then count them. One pass per level. Simple, and many passes.

FP-GROWTH

build a compressed prefix tree of the transactions, then mine it recursively. Two passes in total. Faster, and much more memory.

PCY AND ITS VARIANTS

use the spare memory during pass 1 to hash item pairs into counters, so that pass 2 can skip any pair whose bucket was infrequent. A nice use of memory that was going to sit idle anyway.

Pass count is the design axis along which this family varies — Apriori trades passes for memory, FP-growth trades memory for passes, and PCY recovers some passes using memory that was already free. Which is Module 01 §2's cost model determining the algorithm rather than asymptotic operation counts, and it is the clearest example of that principle in the course.

3 What the rules actually mean

- **It is not causal.** $X \rightarrow Y$ and $Y \rightarrow X$ have exactly the same support and different confidences, and neither of them says anything about which came first — the arrow is notation, not a claim about direction.
- **It may be a confound.** Both items may be driven by a third factor — a season, a promotion that bundled them, a store location, or a customer segment that buys both for unrelated reasons.
- **It may be an artefact of how transactions were defined.** Change the basket boundary — a visit, a day, a session, an order — and the rules change — which is a data-collection decision rather than a finding about behaviour. This is the most practically important caveat, because it means the finding is partly a property of your schema.
- **And the famous examples are mostly unverified.** The beer-and-nappies story, which appears in nearly every introduction to this topic, has no published source and appears to be apocryphal — which is worth knowing given how often it is offered as evidence that the technique works.
- **So a rule is a starting point for an investigation rather than a conclusion** — which is §4's framing and resolves most of the difficulty with this material.

AND THE MULTIPLICITY PROBLEM IS ACUTE HERE

Mining itemsets over a thousand items considers an astronomically large candidate space — and reports every set that passes a support threshold, which is a search of unprecedented breadth conducted automatically.

So the rules returned include whatever passed by chance — and at a low support threshold on a large transaction set, that is a great many of them, with the most striking-looking rules being disproportionately likely to be the chance ones.

Which means the support threshold is doing statistical work it was never designed for: it is a computational pruning parameter being used as a significance filter, and the relationship between a support level and a false discovery rate is not one anybody computed.

So test the surviving rules properly: a permutation test on held-out transactions, with a correction for the number of rules examined (Module 11) — which almost no published application of this technique does, and which is Project 2's requirement.

4 Where this is the right tool

- **Exploratory data understanding** — finding out what co-occurs in a dataset you do not yet know, which is a legitimate and honest use because the output is explicitly a set of leads rather than conclusions.
- **Feature construction**, where a frequent combination becomes a candidate feature for a supervised model that is *then* evaluated properly (CSCE 633) — which moves the validation to where it can be done well.

- **Constraint and dependency discovery** in databases and in logs, where **the patterns are structural rather than behavioural** — a functional dependency that holds in every one of ten million rows is a different kind of finding from a shopping correlation.
- **And sequence and episode mining**, where **the ordering constraint cuts the candidate space substantially and makes the findings more interpretable**, because a temporal ordering at least constrains the causal direction.
- **But for prediction, use a supervised model. Rules are interpretable and are not competitive on accuracy**, and choosing them for predictive performance is a mistake — choose them when the interpretability is the deliverable. **The exploratory use is the honest one**, and **framing the output as hypotheses rather than findings resolves most of this module's difficulties**.

WHAT TO REPORT

Support, confidence, and lift for every rule — because the three together are interpretable and any one of them alone is not (§1), and giving all three costs nothing.

The thresholds you used and how many rules passed them — which is the hypothesis count (Module 11 §1), and is the number that makes the rest interpretable.

The transaction definition, explicitly, since the rules depend on it (§3) — and a reader who does not know your basket boundary cannot assess your rules at all.

And validation on held-out transactions for any rule you intend to act on — which is the step that distinguishes a hypothesis from a finding, and is the whole of this course's position in one sentence.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Tan et al. — Introduction to Data Mining, chapters 5 and 6	The whole module , with the interestingness measures treated more carefully than elsewhere.
Mining of Massive Datasets, chapter 6 (free)	§2's algorithms with the memory analysis — and PCY is explained better here than anywhere.
Agrawal & Srikant — Fast Algorithms for Mining Association Rules (free)	§2's first algorithm in the original , with the closure property stated as the key insight.
Han et al. — Mining Frequent Patterns without Candidate Generation (free)	§2's second algorithm — FP-growth, and the pass-count argument for it.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Compute support, confidence, and lift** for ten rules on a transaction dataset.
- 2 **Sort by confidence and by lift** and compare the top ten lists.
- 3 **Construct a high-confidence rule with lift 1.0** and explain it.
- 4 **Find a high-lift low-support rule** and argue it is noise.
- 5 **Implement Apriori** and count the passes it makes.
- 6 **Implement FP-growth** and compare runtime and memory.
- 7 **Change your transaction definition** and report how the top rules change.
- 8 **Count the rules that passed your threshold** — this is your hypothesis count.
- 9 **Run a permutation test** on your top ten rules.
- 10 **Validate one rule on held-out transactions** and report the result either way.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Define support, confidence, and lift.
- 2 Why can high confidence mean nothing?
- 3 What does a confidence-sorted rule list return?
- 4 What is lift's own weakness?
- 5 State the downward closure property and what it prunes.
- 6 Contrast Apriori, FP-growth, and PCY by pass count and memory.
- 7 Give four reasons a rule means less than it appears to.
- 8 Which caveat is most practically important, and why?
- 9 Why is the support threshold doing the wrong job?
- 10 Give four legitimate uses, and what to use instead for prediction.